



Orientação a Objetos: Aplicação Prática

:::: Introdução a Algoritmos

O que faremos hoje?

- Compreender a descrição de um problema.
- Identificar classes, atributos e métodos.
- Entender como objetos se relacionam.
- Implementar as classes em Python.
- Criar um main utilizando os objetos.
- Resolver um exercício parecido, mas em outro contexto.

O Problema

A lanchonete de uma universidade precisa de um programa para registrar os pedidos realizados pelos estudantes.

Cada cliente possui nome e matrícula. A lanchonete vende diferentes produtos, e cada produto possui nome e preço.

Um pedido pertence a um único cliente e pode conter vários produtos. Como um cliente pode solicitar mais de uma unidade do mesmo produto, o sistema também precisa registrar a quantidade solicitada.

O que o programa deverá fazer?

- Criar clientes e produtos.
- Abrir um pedido para determinado cliente.
- Adicionar produtos e suas quantidades ao pedido.
- Calcular o subtotal de cada item.
- Calcular o valor total do pedido.
- Exibir um resumo completo da compra.

Exemplo: Ana pediu 2 sanduíches, 1 suco e 3 pães de queijo.

Identificando as Classes

Ao ler o problema, encontramos quatro elementos principais:

- Cliente → representa a pessoa que realiza o pedido
- Produto → representa algo vendido pela lanchonete
- ItemPedido → registra um produto e a quantidade solicitada
- Pedido → reúne cliente, itens e cálculo do total

Nem todo substantivo do texto vira classe. Escolhemos classes quando o elemento possui dados próprios, ações próprias ou precisa ser manipulado no código.

Por que criar ItemPedido?

- O produto possui nome e preço, mas não possui uma quantidade fixa.
- O mesmo produto pode aparecer com quantidade 1 em um pedido, quantidade 2 em outro e quantidade 5 em outro.
- Por isso, a quantidade pertence ao item daquele pedido, e não ao produto em si.

Produto: “Sanduíche”, R\$ 18,00

ItemPedido: 2 unidades do produto “Sanduíche”

Atributos, Métodos e Responsabilidades

Sistema de Pedidos da Lanchonete

C Cliente
-nome: str -matricula: str
+Cliente(nome: str, matricula: str) +exibir(): void

C Produto
-nome: str -preco: float
+Produto(nome: str, preco: float) +exibir(): void

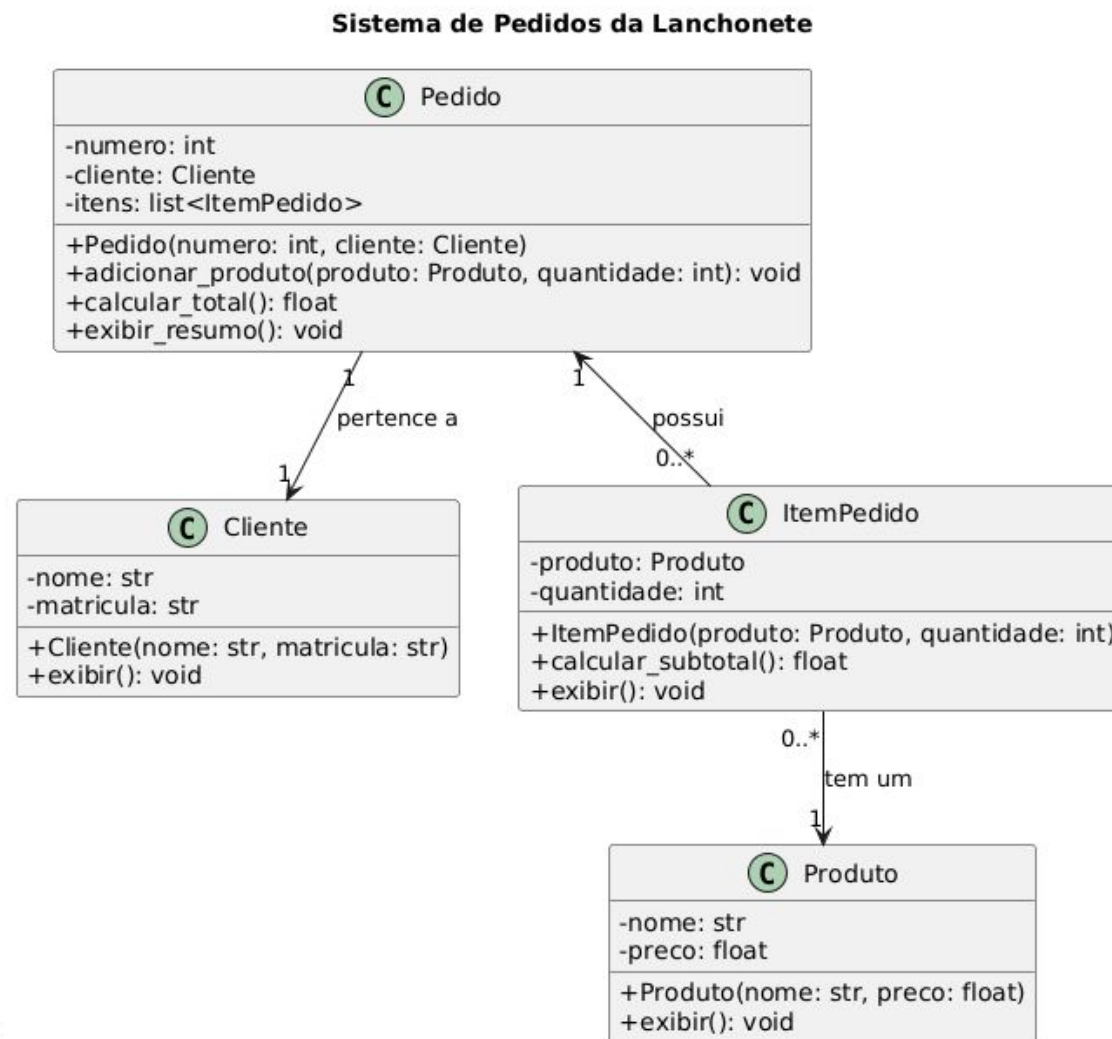
C ItemPedido
-produto: Produto -quantidade: int
+ItemPedido(produto: Produto, quantidade: int) +calcular_subtotal(): float +exibir(): void

C Pedido
-numero: int -cliente: Cliente -itens: list<ItemPedido>
+Pedido(numero: int, cliente: Cliente) +adicionar_produto(produto: Produto, quantidade: int): void +calcular_total(): float +exibir_resumo(): void

Relacionamento entre as Classes

- Um Pedido tem um Cliente.
- Um Pedido tem vários objetos ItemPedido.
- Um ItemPedido tem um Produto.
- O método adicionar_produto() recebe um objeto Produto.

Relacionamento entre as Classes



Como queremos usar as classes?

```
cliente = Cliente("Ana", "20261234")

sanduiche = Produto("Sanduíche", 18.00)
suco = Produto("Suco", 7.50)
pao_queijo = Produto("Pão de queijo", 4.00)

pedido = Pedido(101, cliente)

pedido.adicionar_produto(sanduiche, 2)
pedido.adicionar_produto(suco, 1)
pedido.adicionar_produto(pao_queijo, 3)

pedido.exibir_resumo()
```

Implementando Cliente

```
class Cliente:
    def __init__(self, nome, matricula):
        self.nome = nome
        self.matricula = matricula

    def exibir(self):
        print(self.nome, "-", self.matricula)

cliente = Cliente("Ana", "20261234")
cliente.exibir()
```

Implementando Produto

```
class Produto:
    def __init__(self, nome, preco):
        self.nome = nome
        self.preco = preco

    def exibir(self):
        print(self.nome, "- R$", self.preco)

sanduiche = Produto("Sanduíche", 18.00)
suco = Produto("Suco", 7.50)

sanduiche.exibir()
suco.exibir()
```

Implementando ItemPedido

Um item do pedido reúne um objeto Produto e a quantidade solicitada daquele produto:

```
class ItemPedido:
    def __init__(self, produto, quantidade):
        self.produto = produto
        self.quantidade = quantidade

    def calcular_subtotal(self):
        return self.produto.preco * self.quantidade
```

self.produto guarda o objeto Produto completo, não apenas o nome do produto.

Exibindo um ItemPedido

```
def exibir(self):  
    subtotal = self.calcular_subtotal()  
  
    print(  
        self.quantidade,  
        "x",  
        self.produto.nome,  
        "- R$",  
        subtotal  
    )
```

Como produto é um objeto, podemos acessar `self.produto.nome` e `self.produto.preco`.

Implementando Pedido: início

```
class Pedido:
    def __init__(self, numero, cliente):
        self.numero = numero
        self.cliente = cliente
        self.itens = []

pedido = Pedido(101, cliente)
```

Ao criar o pedido, ele passa a ter um cliente e uma lista vazia de itens.

pedido

- número: 101
- cliente: objeto Cliente
- itens: lista vazia

Pedindo: adicionando produtos

```
def adicionar_produto(self, produto, quantidade):  
    if quantidade <= 0:  
        print("A quantidade deve ser maior que zero.")  
        return  
  
    item = ItemPedido(produto, quantidade)  
    self.itens.append(item)  
  
    print(  
        quantidade,  
        "unidade(s) de",  
        produto.nome,  
        "adicionada(s) ao pedido."  
    )
```


O que acontece ao adicionar um produto?

Quando executamos:

```
pedido.adicionar_produto(sanduiche, 2)
```

- O método recebe o objeto sanduiche e a quantidade 2.
- Um objeto ItemPedido é criado com esses dados.
- Esse objeto é adicionado à lista self.itens.
- O pedido passa a ter mais um item registrado.

Pedido: calculando o total

```
def calcular_total(self):  
    total = 0  
  
    for item in self.itens:  
        total += item.calcular_subtotal()  
  
    return total
```

O pedido percorre a lista de itens e pede para cada item calcular seu próprio subtotal.

Pedido: exibindo o resumo

```
def exibir_resumo(self):  
    print()  
    print("PEDIDO", self.numero)  
    print("Cliente:", self.cliente.nome)  
    print("Matrícula:", self.cliente.matricula)  
    print()  
  
    if len(self.itens) == 0:  
        print("0 pedido está vazio.")  
        return
```

O pedido acessa os dados do cliente armazenado em self.cliente.

Pedido: finalizando o resumo

```
def exibir_resumo(self):  
    ...  
    print("Itens:")  
    for item in self.itens:  
        item.exibir()  
  
    print()  
    print("Total: R$", self.calcular_total())
```

Repare que o pedido não calcula o subtotal diretamente. Ele chama o método `exibir()` de cada `ItemPedido`.

Programa Principal

```
def main():  
    cliente = Cliente("Ana", "20261234")  
  
    sanduiche = Produto("Sanduíche", 18.00)  
    suco = Produto("Suco", 7.50)  
    pao_queijo = Produto("Pão de queijo", 4.00)  
  
    pedido = Pedido(101, cliente)  
  
    pedido.adicionar_produto(sanduiche, 2)  
    pedido.adicionar_produto(suco, 1)  
    pedido.adicionar_produto(pao_queijo, 3)  
  
    pedido.exibir_resumo()  
  
main()
```

Saída Esperada

2 unidade(s) de Sanduíche adicionada(s) ao pedido.
1 unidade(s) de Suco adicionada(s) ao pedido.
3 unidade(s) de Pão de queijo adicionada(s) ao pedido.

PEDIDO 101

Cliente: Ana

Matrícula: 20261234

Itens:

2 x Sanduíche - R\$ 36.0

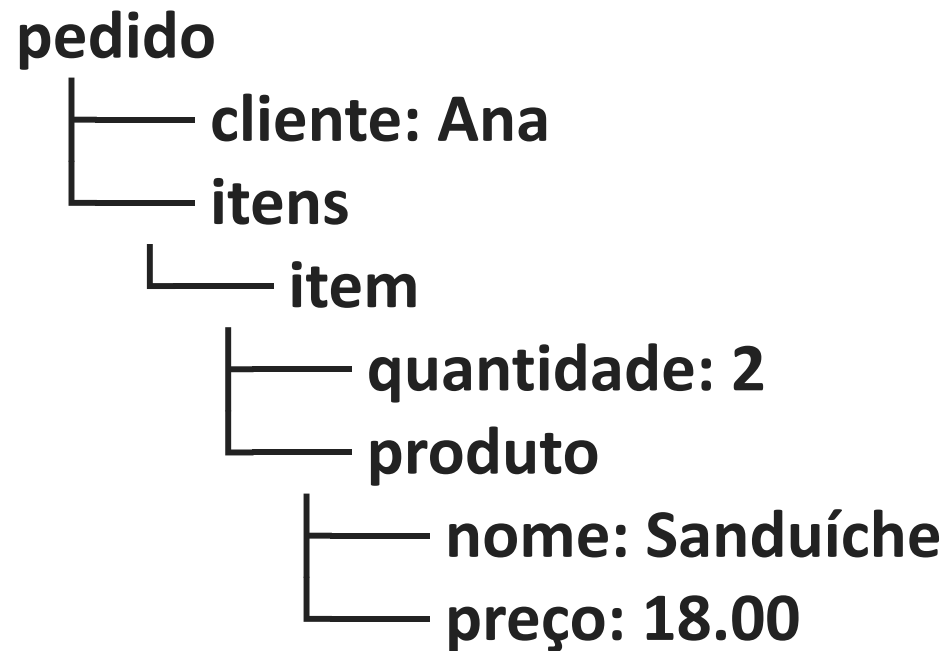
1 x Suco - R\$ 7.5

3 x Pão de queijo - R\$ 12.0

Total: R\$ 55.5

Acompanhando a Estrutura dos Objetos

Depois de executar `pedido.adicionar_produto(sanduche, 2)`, a estrutura fica assim:



Exercício: Biblioteca da Universidade

A biblioteca da universidade precisa de um programa para registrar seus livros e empréstimos.

Cada leitor possui nome e matrícula. Cada livro possui título, autor e uma informação indicando se está disponível.

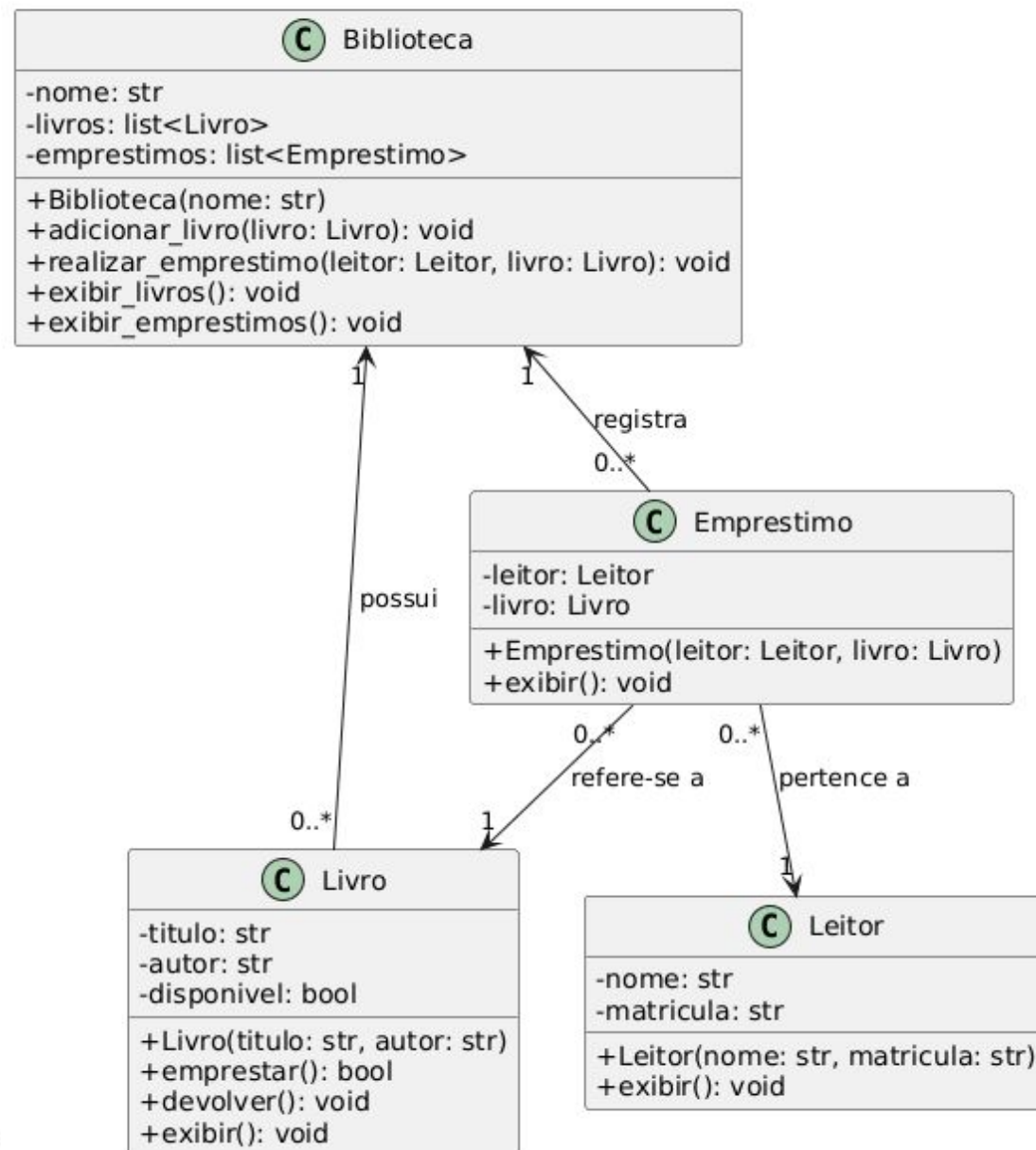
A biblioteca possui vários livros e registra os empréstimos realizados. Cada empréstimo relaciona um leitor e um livro.

Quando um empréstimo é realizado, o livro deixa de estar disponível.

O sistema de Biblioteca Deverá Permitir

- Cadastrar livros na biblioteca.
- Realizar um empréstimo.
- Impedir o empréstimo de um livro indisponível.
- Exibir os livros cadastrados.
- Exibir os empréstimos realizados.

Sistema de Empréstimos da Biblioteca





PUC Minas

© **PUC Minas** • Todos os direitos reservados, de acordo com o art. 184 do Código Penal e com a lei 9.610 de 19 de fevereiro de 1998.

Proibidas a reprodução, a distribuição, a difusão, a execução pública, a locação e quaisquer outras modalidades de utilização sem a devida autorização da Pontifícia Universidade Católica de Minas Gerais.